

✓ BPL_TEST2_Batch script with OMSimulator

The key library OMSimulator is installed.

After the installation a small application BPL_TEST2_Batch is loaded and run. You can continue with this example if you like.

```
!lsb_release -a # Actual VM Ubuntu version used by Google
```

```
No LSB modules are available.  
Distributor ID: Ubuntu  
Description:   Ubuntu 24.04.5 LTS  
Release:      24.04  
Codename:     noble
```

```
!python --version
```

```
Python 3.13.15
```

```
!pip install OMSimulator
```

```
Collecting OMSimulator  
  Downloading omsimulator-3.0.0.post116.tar.gz (7.6 kB)  
  Installing build dependencies ... done  
  Getting requirements to build wheel ... done  
  Preparing metadata (pyproject.toml) ... done  
Requirement already satisfied: requests in /usr/local/lib/python3.13/d  
Requirement already satisfied: lxml>=5.3.1 in /usr/local/lib/python3.1  
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/  
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.  
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/py  
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/py  
Building wheels for collected packages: OMSimulator  
  Building wheel for OMSimulator (pyproject.toml) ... done  
  Created wheel for OMSimulator: filename=omsimulator-3.0.0.post116-py  
  Stored in directory: /root/.cache/pip/wheels/eb/b0/ee/5194adbcfe75a2  
Successfully built OMSimulator  
Installing collected packages: OMSimulator  
Successfully installed OMSimulator-3.0.0.post116
```

```
!pip install DyMat
```

```
Collecting DyMat  
  Downloading DyMat-0.7.tar.gz (24 kB)  
  Preparing metadata (setup.py) ... done  
Building wheels for collected packages: DyMat  
  Building wheel for DyMat (setup.py) ... done  
  Created wheel for DyMat: filename=dymat-0.7-py3-none-any.whl size=19  
  Stored in directory: /root/.cache/pip/wheels/bd/8e/7a/d63cada988615a  
Successfully built DyMat
```

```
Installing collected packages: DyMat
Successfully installed DyMat-0.7
```

```
!pip install git+https://github.com/janpeter19/FMU_explore.git
```

```
Collecting git+https://github.com/janpeter19/FMU_explore.git
  Cloning https://github.com/janpeter19/FMU_explore.git to /tmp/pip-re
  Running command git clone --filter=blob:none --quiet https://github.
  Resolved https://github.com/janpeter19/FMU_explore.git to commit 47d
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Building wheels for collected packages: fmu_explore
  Building wheel for fmu_explore (pyproject.toml) ... done
  Created wheel for fmu_explore: filename=fmu_explore-1.2.5-py3-none-a
  Stored in directory: /tmp/pip-ephem-wheel-cache-9rrh1b33/wheels/98/3
Successfully built fmu_explore
Installing collected packages: fmu_explore
Successfully installed fmu_explore-1.2.5
```

✓ BPL_TEST2_Batch setup

Now specific installation and the run simulations. Start with connecting to Github.
Then upload the two files:

- FMU - BPL_TEST2_Batch_linux_om_me.fmu
- Setup-file - BPL_TEST2_Batch_fmпы_explore.py

```
%bash
git clone https://github.com/janpeter19/BPL_TEST2_Batch
```

```
Cloning into 'BPL_TEST2_Batch'...
```

```
%cd BPL_TEST2_Batch
```

```
/content/BPL_TEST2_Batch
```

```
# Import the application model and context variables
from BPL_TEST2_Batch_oms import*
```

```
Linux - run FMU pre-compiled OpenModelica
```

```
# Import the general FMU_explore functions ana adapt them to the appl
from fmu_explore_oms import AdaptWith
```

```
Application = AdaptWith(parValue, parLocation, parCheck, \
                        fmu_model, fmu_process_diagram, \
                        MSL_usage, MSL_version, BPL_version, \
                        simulationTime, timeDiscreteStates, \
```

```
diagrams, ax, lines)
```

```
par = Application.par
init = Application.init
restart = Application.restart
simu = Application.simu
show = Application.show

resetPen = Application.resetPen
process_diagram = Application.process_diagram

describe_MSL = Application.describe_MSL
FMU_explore_info = Application.FMU_explore_info
system_info = Application.system_info
SDG = Application.SDG
```

```
run -i BPL_TEST2_Batch_oms_explore.py
```

Model for the process has been setup. Key commands:

- par() - change of parameters and initial values
- init() - change initial values only
- simu() - simulate and plot
- newplot() - make a new plot
- show() - show plot from previous simulation
- disp() - display parameters and initial values from the last s
- describe() - describe culture, broth, parameters, variables with v

Note that both disp() and describe() takes values from the last simulation and the command process_diagram() brings up the main configuration

Brief information about a command by help(), eg help(simu)
Key system information is listed with the command system_info()

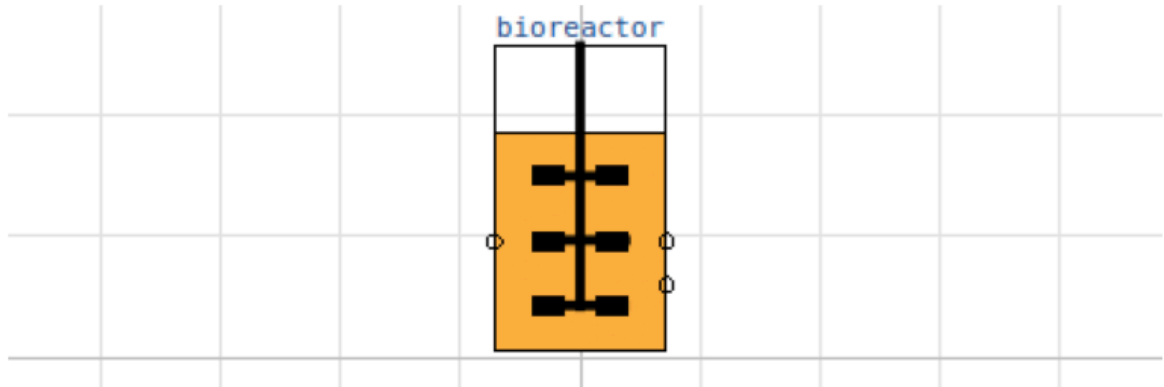
```
%matplotlib inline
plt.rcParams['figure.figsize'] = [25/2.54, 20/2.54]
```

```
import warnings
warnings.filterwarnings("ignore")
```

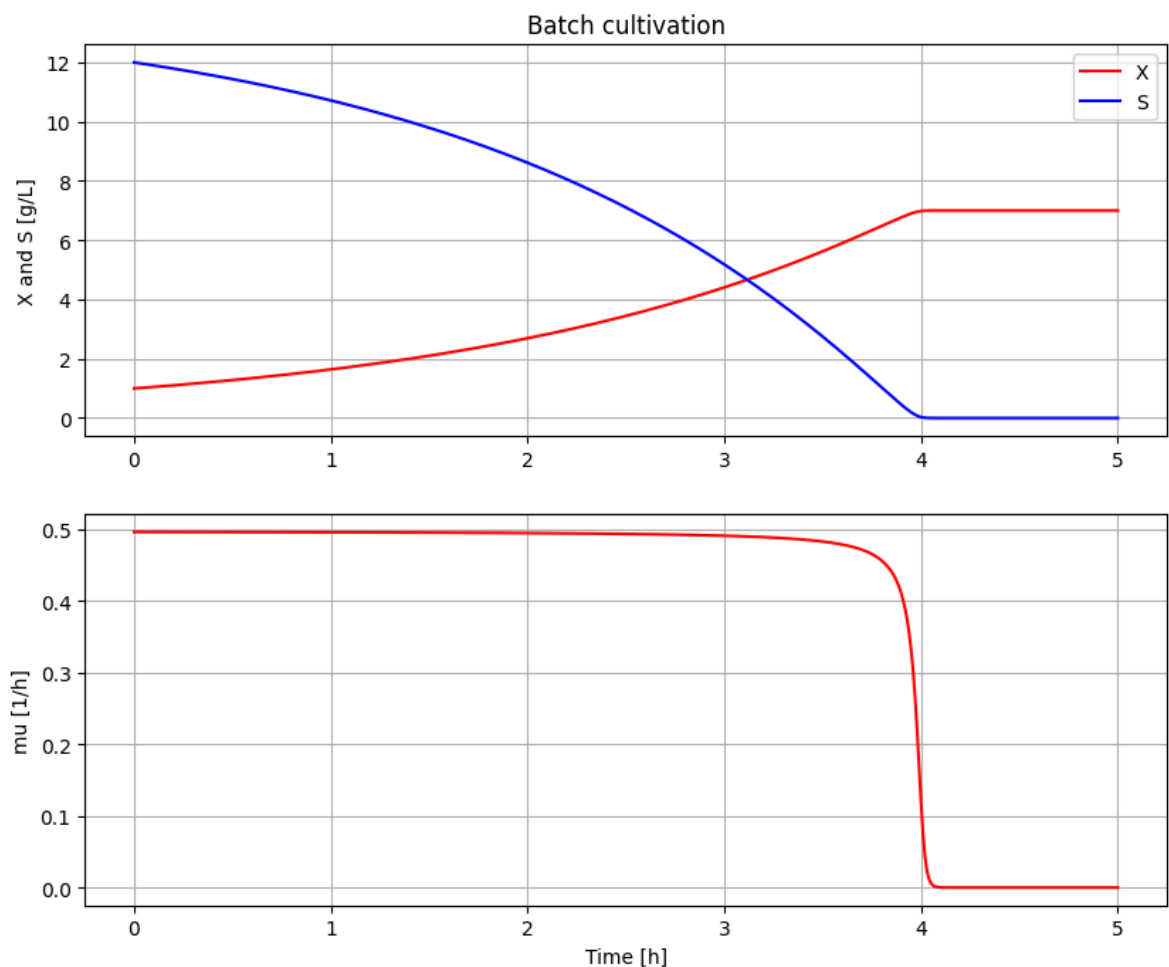
✓ BPL_TEST2_Batch - demo

```
process_diagram()
```

No processDiagram.png file in the FMU, but try the file on disk.

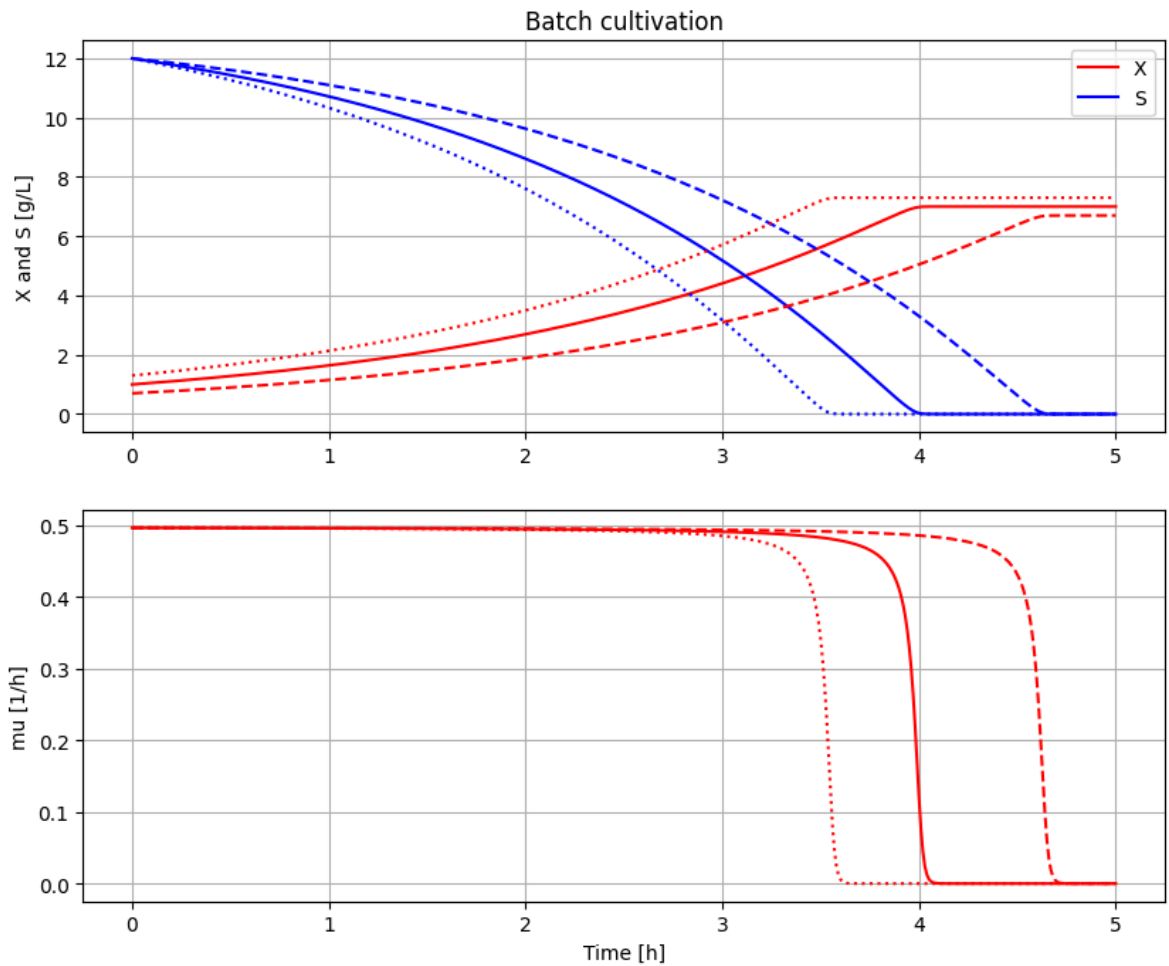


```
# Simulation with default values of the process
newplot(plotType='TimeSeries')
simu()
```



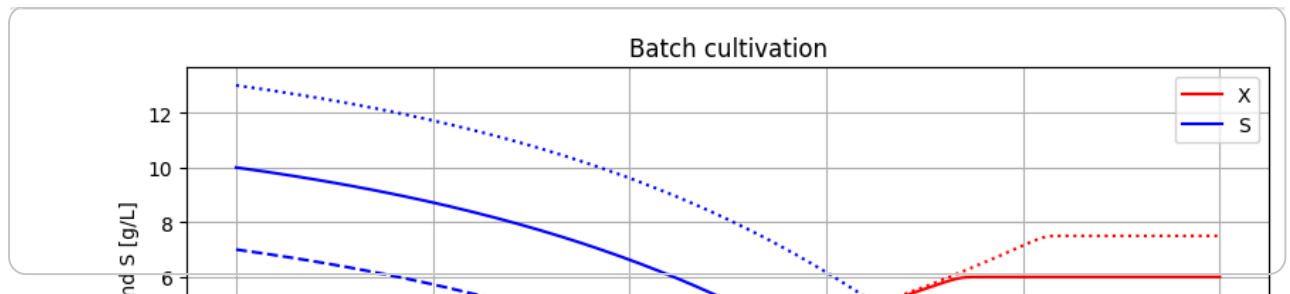
```
# Simulation were initial value of biomass VX_0 is varied
newplot(plotType='TimeSeries')
for value in [1.0, 0.7, 1.3]:
    init(VX_start=value)
    simu()

# Restore default value of VX_start
init(VX_start=1.0)
```



```
# Simulation were initial value of substrate VS_0 is varied
newplot(plotType='TimeSeries')
for value in [10, 7, 13]:
    init(VS_start=value)
    simu()

# Restore default value of VS_start
init(VS_start=10)
```



Start coding or [generate](#) with AI.

```
!lsb_release -d
```

No LSB modules are available.
Description: Ubuntu 24.04.5 LTS

